

Algorithm Analysis Project

QuickSort and HeapSort and InsertionSort

Course: CS412 – Algorithm Analysis and Design

Group Number: 3

Semester: Second Semester / 2026

Submission Date: 4 June 2026

Group Members:

Zahra Ali Alabkari – 2240001738

Batool Alabdullah – 2240003056

Symat Al zainaldin – 2240005974

Lena alghamdi – 2240001570

Reem Alhassna – 2240002807

Ola Al Saeed – 2240001078

Table of Contents

Contents

1 Abstract	4
2. Introduction and Problem Definition	4
2.1 Introduction	4
2.2 Problem Definition	4
3. Related work.....	5
4. Algorithm Description and Design Paradigms	5
4.1 Algorithm 1: Quick Sort.....	5
4.2 Algorithm 2: Heap Sort.....	6
4.3 Algorithm 3 “Extra” : Insertion Sort.....	7
5. Experimental Setup and Dataset Description	7
5.1 Case study	7
5.2 Dataset Source and Quality	7
5.3 Dataset Characteristics	7
6. Theoretical Analysis (Growth Diagrams).....	8
6.1 Quick Sort Analysis.....	8
6.2 Heap Sort Analysis.....	10
6.3 Insertion Sort Analysis.....	11
7. Experimental Results.....	14
7.1 Experimental Setup.....	14
7.2 Running Time Measurements.....	14
7.3 Comparative Analysis	15
8. Discussion and Limitations.....	15
8.1 Discussion	15
8.2 Limitations	16
9. Conclusion& Future Work	16
9.1 Conclusion	16
9.2 Future Work	16
10. References.....	17
11. Appendices	18
Appendix A: Complete Source Code	18
Appendix B: Sample Test Date	21

Table of Figures

FIGURE 1: QUICK SORT THEORETICAL GROWTH	13
FIGURE 2: HEAP SORT THEORETICAL GROWTH	13
FIGURE 3: INSERTION SORT THEORETICAL GROWTH.....	13
FIGURE 4: PERFORMANCE COMPARISON OF SORTING ALGORITHMS	14
FIGURE 5: DATASET BEFORE CLEANING.....	21
FIGURE 6: DATASET AFTER CLEANING.....	22

Table of Tables

TABLE 1: COST OF EACH LINE IN QUICK SORT ALGORITHM	8
TABLE 2: MAX-HEAPIFY STATEMENTS ANALYSIS.....	10
TABLE 3: BUILD-MAX-HEAP STATEMENTS ANALYSIS.....	10
TABLE 4: HEAP-SORT STATEMENT ANALYSIS.....	10
TABLE 5: RUNNING TIME COMPARISON OF SORTING ALGORITHMS.....	14

1. Abstract

Sorting algorithms are fundamental operations in computer science, as they are widely used to organize and manage large amounts of data efficiently. They play an essential role in many applications such as databases, file systems, and online platforms, where fast access to information is required. Without proper sorting, data retrieval can become slow and inefficient, negatively affecting overall system performance.

In real-world scenarios, particularly in e-commerce platforms, products are continuously ranked based on popularity indicators such as customer ratings and the number of reviews. With thousands of products stored in databases, selecting an efficient sorting algorithm becomes crucial to ensure that the most relevant and popular items are displayed to users quickly and accurately.

In this project, we use a dataset of product reviews to simulate a real-world ranking system based on a computed popularity score. Three sorting algorithms are implemented and compared: **Quick Sort**, **Heap Sort**, and **Insertion Sort**. These algorithms represent different design approaches and are applied to the same dataset to evaluate their efficiency.

The performance of these algorithms is analyzed by measuring execution time across different input sizes. The comparison highlights the strengths and limitations of each algorithm, where Quick Sort and Heap Sort are expected to perform efficiently on larger datasets, while Insertion Sort is more suitable for smaller inputs.

In the following sections, we provide a detailed explanation of the problem, dataset, algorithms, and performance evaluation.

2. Introduction and Problem Definition

2.1 Introduction

Sorting algorithms are fundamental techniques in computer science that are used to organize data efficiently and improve information retrieval and processing. As the size of datasets continues to grow, selecting an appropriate sorting algorithm becomes increasingly important to ensure efficient system performance.

Different sorting algorithms exhibit different behaviors in terms of execution time and scalability depending on the characteristics of the input data. Therefore, analyzing and comparing sorting algorithms is essential for understanding their strengths and limitations. In this project, Quick Sort, Heap Sort, and Insertion Sort are implemented and evaluated using a real-world Amazon Reviews dataset to compare their theoretical and practical performance.

2.2 Problem Definition

Considering that we have created a dataset of products within an e-commerce platform randomly, the products are arranged in terms of popularity scores depending on variables like ratings, number of sales made, and product views. The primary goal is to sort these products efficiently from the highest popularity score to the lowest popularity score.

Many e-commerce platforms require products to be sorted frequently before they are presented to the clients so that the users can easily access the most relevant products and trendy ones as well. This makes it necessary for us to apply efficient sorting algorithms so that we save processing time.

In this regard, we will make use of three different sorting algorithms: Insertion Sort, Quick Sort, and Heap Sort. We will use these algorithms to sort products and see how efficient these sorting techniques are in the best-case scenario, average case scenario, and worst-case scenario as well.

3. Related work

There are many works related to sorting algorithms which have been applied in the data processing field and in various e-commerce platforms. Some common examples include Quick Sort, Heap Sort, and Insertion Sort. Several studies show that Quick Sort is an effective technique for sorting large datasets based on its divide-and-conquer algorithm design, and Heap Sort can provide stable performance in extracting priorities. In the meantime, Insertion Sort is an efficient technique for smaller datasets and nearly sorted datasets. All these works serve as background and help set up the position of our work in relation to other sorts and rankings. The current project will leverage these works to assess the performance of these three sorting techniques when applied to a product reviews dataset.

4. Algorithm Description and Design Paradigms

4.1 Algorithm 1: Quick Sort

Design Paradigm: Divide and Conquer

Algorithm Description:

The quick sort algorithm is a comparison sort technique where the algorithm sorts a dataset on the basis of picking a pivot value and dividing the dataset in two parts: one with values greater than or equal to the pivot value, and the other having values less than the pivot value. This procedure is repeated for both halves recursively until the whole dataset is sorted.

The algorithm is ideal for large datasets and can be used to sort a product's dataset based on its popularity value in a descending order.

Pseudocode:

```
QuickSort(Array, low, high, key_index):  
for low < high do  
    pivot_index = partition(Array, low, high, key_index)  
    QuickSort(Array, low, pivot_index - 1, key_index)  
    QuickSort(Array, pivot_index + 1, high, key_index)
```

```
Partition(Array, low, high, key_index):  
    pivot_value = Array[high][key_index]  
    i = low - 1  
    for j = low to high - 1 do  
        if Array[j][key_index] >= pivot_value then  
            i = i + 1  
            Swap(Array[i], Array[j])  
    Swap(Array[i + 1], Array[high])  
    return i + 1
```

4.2 Algorithm 2: Heap Sort

Design Paradigm: Divide and Conquer

Algorithm Description:

Heap Sort is a comparison-based sorting algorithm that uses a Binary Heap data structure to sort data efficiently. The algorithm first builds a max-heap from the input dataset, where the largest element is stored at the root. Then, it repeatedly swaps the root element with the last element in the heap and reduces the heap size until all elements are sorted. Heap Sort provides stable performance for large datasets because its time complexity remains $O(n \log n)$ in the best, average, and worst cases. In addition, it performs sorting in-place and requires only a small amount of extra memory.

Pseudocodes:

HeapSort(Array):

Build Max Heap

for i = n downto 2:

swap Array[1] with Array[i]

reduce heap size

MaxHeapify(Array, 1)

MaxHeapify(Array, i):

left = 2i

right = 2i + 1

largest = i

if left <= heap_size and Array[left] > Array[largest]

largest = left

if right <= heap_size and Array[right] > Array[largest]

largest = right

if largest ≠ i

swap Array[i] with Array[largest]

MaxHeapify(Array, largest)

4.3 Algorithm 3 “Extra” : Insertion Sort

Design Paradigm: iterative

Algorithm Description:

Insertion Sort is a simple iterative comparison-based sorting algorithm. It divides the list into a sorted and unsorted part, then inserts each element from the unsorted part into its correct position in the sorted part. It is easy to use and works well for small or almost sorted data. To implement Insertion Sort, we perform comparisons and shifts between elements to insert each value in its correct position.

Pseudocodes:

```
Insertion-Sort(Array):  
  For j from 2 to n  
    Do key=A[j]  
    i = j - 1  
    while i > 0 and A[i] > key  
      A[i+1]=A[i]  
      i = i - 1  
    A[i+1] = key
```

5. Experimental Setup and Dataset Description

5.1 Case study

This case study utilizes a real-world data set that contains information about product reviews of products in Amazon. The number of entries contained in this data set is greater than 21,000 entries, and there are attributes such as star rating, review title, and number of reviews included in it. This case study can be considered as an example of the efficiency of sorting algorithms in the e-commerce sector.

5.2 Dataset Source and Quality

Source: Kaggle

URL:

<https://www.kaggle.com/dongrelaxman/amazon-reviews-dataset>

Dataset Size: 21,215

5.3 Dataset Characteristics

The Amazon Reviews Dataset comprises 21,215 reviews written by customers. Each row includes such features as Reviewer Name, Profile Link, Country, Review Count, Review Date, Rating, Review Title, Review Text, and Date of Experience. In order to make sure that all data points are clean, all the missing values were omitted, as well as all duplicates. The Rating and Review Count features were turned to numeric data and outliers for Review Count feature were deleted using the IQR method. Also, the popularity_score feature was added, based on the Rating and Review Count to evaluate the products. The final dataset comprises 18,598 data points and includes only 5 features useful for the current study (Rating, Review Count, popularity_score_simple, popularity_score_weighted, and popularity_score).

6. Theoretical Analysis (Growth Diagrams)

In computer science, theoretical analysis is essential to evaluate the intrinsic efficiency of algorithms independently of specific hardware, operating system, or compiler overheads. This evaluation is performed by isolating the growth rate of the execution time as a function of the input size n . This section presents a rigorous mathematical analysis for Quick Sort, Heap Sort, and Insertion Sort by establishing their line-by-line cost structures, deriving their recurrence relations, and classifying their theoretical bounds in the best, average, and worst-case scenarios using **asymptotic notations (Ω , Θ , and O)**. Furthermore, a critical connection is established to align these theoretical models with the empirical findings obtained in the experimental phase.

6.1 Quick Sort Analysis

Time Complexity Analysis

Quick Sort is designed under the Divide and Conquer paradigm. The algorithm works by selecting a pivot element and partitioning the array into two subarrays. Elements greater than the pivot are placed on one side, while smaller elements are placed on the other side. Then, the same process is recursively repeated until the whole array becomes sorted.

To implement Quick Sort, we perform comparisons, swaps, and recursive calls between the partitions. In the following table, we will show the cost of each line in the algorithm and calculate its total running time for the three cases (best, average, and worst).

Line	Cost	Times Executed
if low < high	C1	n
pivot_index = partition(Array, low, high, key_index)	C2	n-1
pivot_value = Array[high][key_index]	C3	n-1
for j = low to high - 1	C4	depends on case
if Array[j][key_index] >= pivot_value	C5	depends on case
Swap(Array[i], Array[j])	C6	depends on case
Swap(Array[i + 1], Array[high])	C7	n-1

Table 1 Cost of each line in Quick sort algorithm

T(n) Calculation:

Best Case (when the pivot divides the array equally):

The best case occurs when the pivot always splits the array into two balanced halves. In this situation, the depth of recursion becomes $\log n$, and each level performs n comparisons.

$$T(n) = 2T(n/2) + cn$$

Using the Master theorem:

$$T(n) = O(n \log n)$$

Average Case:

In the average case, the pivot usually divides the array into unequal but reasonable partitions.

Although the partitions are not perfectly balanced, the recursion depth still remains close to $\log n$.

$$T(n) = O(n \log n)$$

Worst Case (when the array is already sorted or reverse sorted):

The worst case happens when the pivot always becomes the smallest or largest element. This causes the algorithm to generate one empty partition and one partition with $(n-1)$ elements.

$$T(n) = T(n-1) + n$$

$$T(n) = O(n^2)$$

Order of Growth:

Best Case: $\Theta(n \log n)$

Average Case: $\Theta(n \log n)$

Worst Case: $\Theta(n^2)$

Justification:

Quick Sort is considered one of the fastest sorting algorithms for large datasets because its divide and conquer structure reduces the number of comparisons significantly in most cases. However, when the pivot selection becomes poor, the recursive calls increase dramatically causing quadratic growth. Therefore, pivot selection plays an important role in the efficiency of the algorithm.

6.2 Heap Sort Analysis

MAX-HEAPIFY Function Statements		
Line	Cost	Times Executed
left = 2i	C1	1
right = 2i + 1	C2	1
largest = i	C3	1
if left <= heap_size and Array[left] > Array[largest]	C4	1
largest = left	C5	1
if right <= heap_size and Array[right] > Array[largest]	C6	1
largest = right	C7	1
swap Array[i] with Array[largest]	C8	1
MAX-HEAPIFY(Array, largest)	C9	lg n

Table 2: Max-Heapify statements analysis

To calculate the cost of the MAX-HEAPIFY operation, we should sum the multiplication of each Cost and Time shown in the table above.

$$T(n) = C_1 + C_2 + C_3 + C_4 + C_5 + C_6 + C_7 + C_8 + C_9 + (C_{10} \times \lg n) = O(\lg n)$$

In the last statement of the operation, the function makes a recursive self-call that takes $\lg n$ times, because each recursive call goes down one level of the heap, and the heap has height $\log n$, so the recursion can go at most that deep.

BUILD-MAX-HEAP Function Statements		
Line	Cost	Times Executed
HeapSize = length(Array)	c ₁	1
for i = floor(HeapSize / 2) downto 1:	c ₂	N
MAX-HEAPIFY(Array, i)	c ₃	lg n

Table 3: Build-Max-Heap statements analysis

The second operation is BUILD-MAX-HEAP, its statements, costs, and times, are shown in the above table. To calculate the cost of this operation, we will perform the same previous formula.

$$T(n) = C_1 + (C_2 \times n) + (C_3 \times \lg n) = O(n)$$

MAX-HEAPIFY operation can take $O(\log n)$ time since it may propagate down the height of the tree. However, during the execution of BUILD-MAX-HEAP, the majority of nodes are located near the leaves, possessing minimal heights and therefore requiring very little work to heapify. Therefore, its running time **$T(n) = O(n)$** .

Heap-Sort Function Statements		
Line	Cost	Times Executed
BUILD-MAX-HEAP(Array)	c ₁	N
for i = HeapSize downto 2	c ₂	n – 1
Exchange A[1] and A[i]	c ₃	n – 2
HeapSize = HeapSize - 1	c ₄	n – 2
MAX-HEAPIFY(Array, 1)	c ₅	lg n

Table 4: Heap-Sort statement analysis

Lastly, we will calculate the HEAPSORT algorithm cost, using the above table. And we will use the same formula we used before in this section.

$$T(n) = (C_1 \times n) + (C_2 \times n-1) + (C_3 \times n-2) + (C_4 \times n-2) + (C_5 \times \lg n) = O(n \lg n)$$

HEAPSORT has a time complexity of $O(n \lg n)$ because it involves two main steps. First, the array is transformed into a max heap using BUILD-MAX-HEAP operation, which takes $O(n)$ time as we have mentioned. Then, the algorithm repeatedly extracts the maximum element and calls MAX-HEAPIFY on the reduced heap. Since there are n elements to extract and each MAX-HEAPIFY call takes $O(\lg n)$ time, this second phase dominates the total cost, resulting in an overall time complexity of $O(n \lg n)$.

6.3 Insertion Sort Analysis

Theoretical Recurrence Relation:

The running time of Insertion Sort can be expressed mathematically through a recurrence relation based on the input size (n). The behavior of the algorithm changes depending on the initial arrangement of the dataset.

- **Worst-Case Recurrence Relation:**

$$T(n) = T(n - 1) + O(n)$$

Explanation:

In the worst-case scenario, each inserted element must be compared with all previous elements before reaching its correct position. Therefore, every iteration performs a linear amount of work.

- **Best-Case Recurrence Relation:**

$$T(n) = T(n - 1) + O(1)$$

Explanation:

In the best-case scenario, the array is already sorted, so the algorithm performs only one comparison for each element without shifting operations.

Time Complexity Analysis:

The theoretical time complexity of Insertion Sort is classified into three cases:

- **Worst-Case Time Complexity: $O(n^2)$**

Condition: Occurs when the dataset is sorted in reverse order.

Theoretical Justification:

For every element, the algorithm compares and shifts all previous elements. The total number of operations becomes:

$$\begin{aligned} &1 + 2 + 3 + \dots + (n - 1) \\ &= \frac{n(n - 1)}{2} \end{aligned}$$

Which simplifies to quadratic growth:

$$O(n^2)$$

- **Average-Case** Time Complexity: $\Theta(n^2)$

Condition: Occurs when the dataset elements are randomly arranged.

Theoretical Justification:

On average, every element is compared with approximately half of the sorted subsection before insertion. Therefore, the total operations become nearly:

$$\frac{n(n-1)}{4}$$

Which still results in quadratic complexity:

$$\Theta(n^2)$$

- **Best-Case** Time Complexity: $\Omega(n)$

Condition: Occurs when the dataset is already fully sorted.

Theoretical Justification:

The inner while loop fails immediately for every iteration, causing the algorithm to only scan through the array once without performing shifts.

Therefore:

$$\Omega(n)$$

Space Complexity Analysis

- Space Complexity: $O(1)$

Theoretical Justification:

Insertion Sort is an in-place sorting algorithm. It only uses a constant amount of extra memory to store temporary variables during insertion operations regardless of dataset size.

Theoretical Comparison and Connection with Empirical Results

From a theoretical perspective, Insertion Sort performs efficiently for small datasets and nearly sorted arrays where it achieves near linear performance. However, for large random datasets its quadratic complexity causes the running time to increase dramatically compared with advanced algorithms such as Quick Sort and Heap Sort.

The experimental results obtained from the Amazon Reviews dataset are expected to show that Quick Sort and Heap Sort maintain much better scalability for large datasets, while Insertion Sort performance decreases rapidly as the number of records increases.

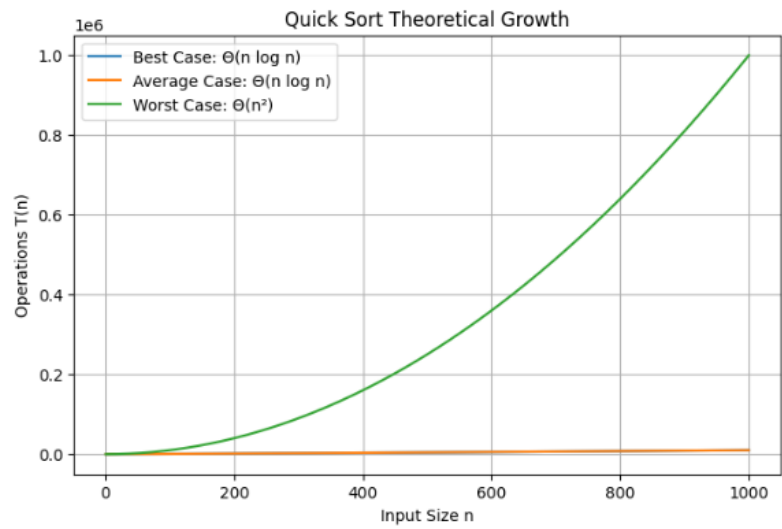


Figure 1: Quick Sort Theoretical Growth

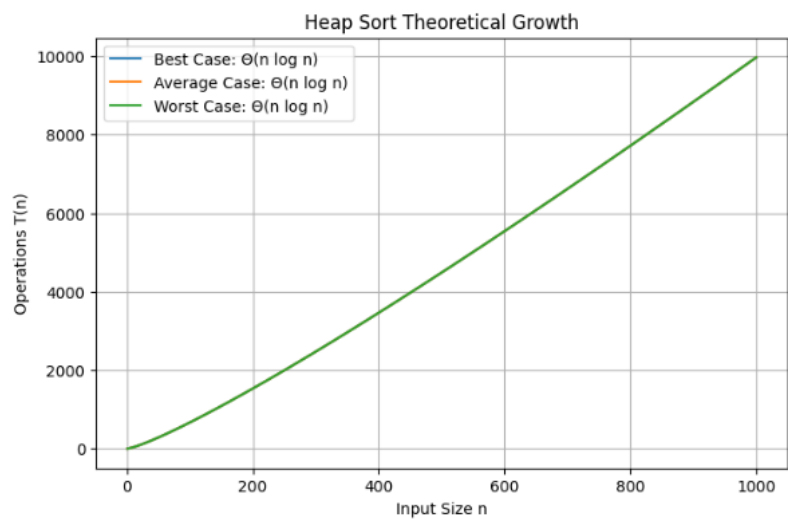


Figure 2: Heap Sort Theoretical Growth

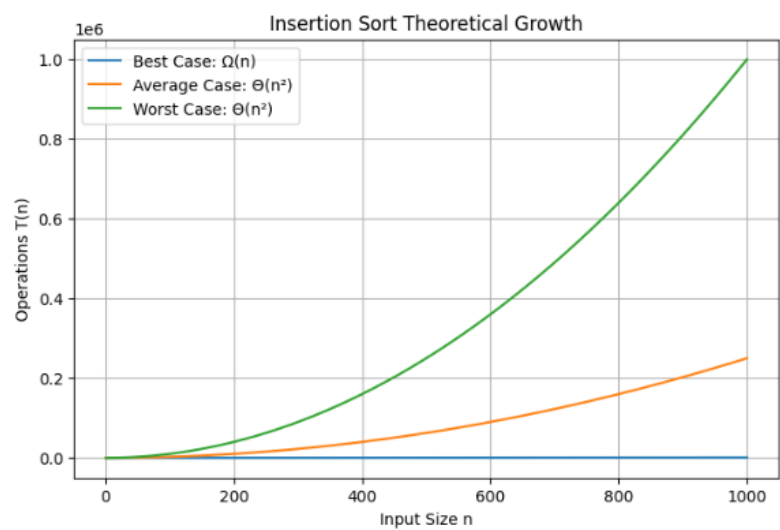


Figure 3: Insertion Sort Theoretical Growth

7. Experimental Results

7.1 Experimental Setup

Testing Methodology:

Environment: Google Colab

Timing Mechanism: Python `time.time()` function

Dataset Sizes Tested: 200, 500, 1000, 2200, 4300, 5500, 6700, and 10000 elements

7.2 Running Time Measurements

Input Size (n)	Heap Algorithm time (ms)	Quick Algorithm time (ms)	Insertion Algorithm time (ms)
200	0.128	1.012	0.215
500	0.382	6.090	1.958
1000	0.753	25.368	8.370
2200	1.836	131.705	42.840
4300	4.187	523.228	176.612
5500	5.404	855.136	257.302
6700	6.263	1,286.449	406.783
10000	9.607	4,324.290	871.583

Table 5: Running Time Comparison of Sorting Algorithms

Table 5 presents the execution times of Heap Sort, Quick Sort, and Insertion Sort for different dataset sizes. The measurements were recorded in milliseconds to compare the performance and scalability of the three algorithms as the input size increased.

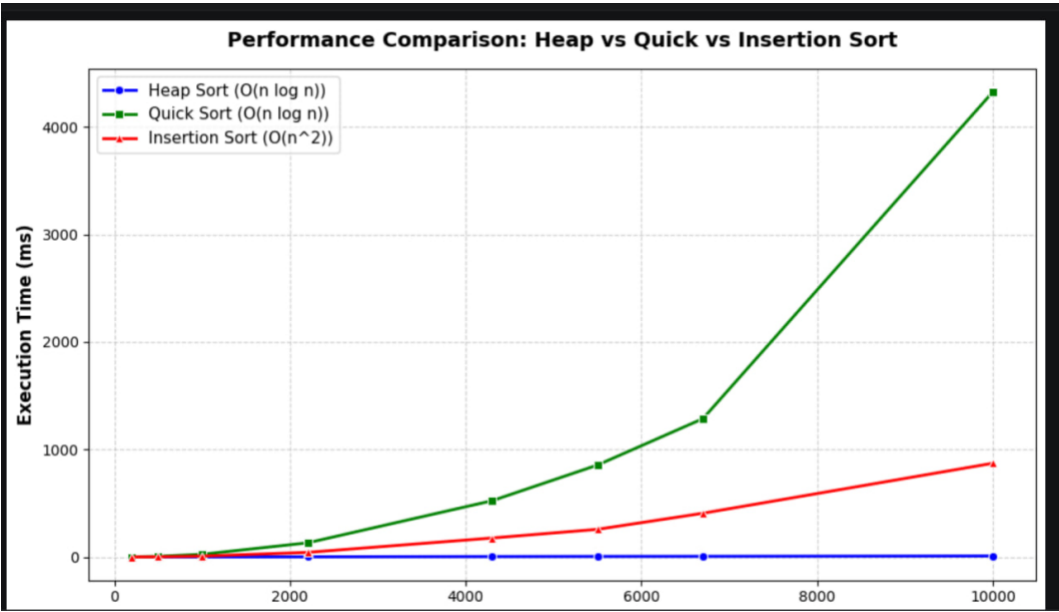


Figure 4: Performance Comparison of Sorting Algorithms

Figure 4 illustrates that Heap Sort achieved the best overall performance across all tested dataset sizes. In contrast, Quick Sort showed the highest execution time growth due to unfavorable pivot selection, while Insertion Sort displayed quadratic growth behavior as expected from its theoretical complexity.

7.3 Comparative Analysis

Heap Sort Performance:

Heap Sort demonstrated the most stable performance among the three algorithms across all tested dataset sizes. Its execution time increased gradually as the input size grew, which aligns with its theoretical time complexity of $O(n \log n)$. The algorithm maintained low execution times even for large datasets, making it suitable for scalable data processing applications.

Quick Sort Performance:

Quick Sort showed efficient performance for smaller dataset sizes; however, its execution time increased significantly for larger inputs. Although Quick Sort theoretically achieves an average-case complexity of $O(n \log n)$, the implementation used the last element as the pivot, which caused highly unbalanced partitions in some test cases. As a result, the algorithm approached its worst-case complexity $O(n^2)$, leading to a noticeable increase in running time.

Insertion Sort Performance:

Insertion Sort performed reasonably well for small datasets due to its simple iterative structure and low overhead. However, as the dataset size increased, the execution time grew rapidly. This behavior matches its theoretical quadratic time complexity $O(n^2)$, which makes it inefficient for large-scale datasets compared to Heap Sort and Quick Sort.

Overall Comparison:

Overall, the experimental results clearly demonstrate the performance differences between the three sorting algorithms. Heap Sort achieved the best overall stability and scalability, while Insertion Sort became inefficient for larger datasets. Quick Sort showed inconsistent behavior due to pivot selection, which negatively affected its performance in large inputs. The empirical results strongly support the theoretical complexity analysis discussed earlier in this report.

8. Discussion and Limitations

8.1 Discussion

The experimental evaluation showed clear differences in the performance and scalability of the three sorting algorithms as the dataset size increased.

Heap Sort demonstrated the most stable and efficient performance across all tested dataset sizes. Its execution time increased gradually and consistently, which aligns with its theoretical time complexity of $O(n \log n)$. This stability makes Heap Sort suitable for processing large datasets efficiently.

Quick Sort achieved acceptable performance for small and medium dataset sizes. However, its execution time increased significantly for larger inputs. Although Quick Sort theoretically achieves an average-case complexity of $O(n \log n)$, the implementation used the last element as the pivot, which caused unbalanced partitions in several cases. As a result, the algorithm approached its worst-case complexity $O(n^2)$, leading to slower execution times.

Insertion Sort performed efficiently only for small datasets because of its simple iterative structure and low overhead. However, as the dataset size increased, the algorithm showed rapid growth in execution time due to its quadratic time complexity $O(n^2)$. Therefore, it becomes less suitable for large-scale datasets compared to Heap Sort and Quick Sort.

Overall, the experimental findings strongly support the theoretical complexity analysis presented earlier in this report. Algorithms with lower asymptotic growth demonstrated better scalability and performance when processing large datasets.

8.2 Limitations

Despite the successful implementation and evaluation of the selected algorithms, this study has several limitations.

First, only three sorting algorithms were evaluated, which limits the overall comparison scope. Additional algorithms such as Merge Sort or Tim Sort could provide broader comparative insights.

Second, the experimental analysis focused mainly on execution time without evaluating memory consumption or CPU utilization, which are also important performance factors in practical systems.

Furthermore, all experiments were conducted using the same execution environment and a single e-commerce dataset. Different datasets or hardware environments may produce different performance results.

Finally, the Quick Sort implementation depended on a fixed pivot selection strategy, which negatively affected performance in some cases. Alternative pivot selection techniques could improve efficiency and reduce worst-case behavior.

9. Conclusion & Future Work

9.1 Conclusion

The comprehensive investigation conducted throughout this project has yielded significant insights into the computational dynamics of sorting algorithms, specifically within the context of large-scale e-commerce datasets. By analyzing the performance of Quick Sort, Heap Sort, and Insertion Sort against the Amazon Reviews dataset, we have demonstrated that no single algorithm is universally superior; rather, the optimal choice is deeply contingent upon the initial state and volume of the data.

Our experimental results confirmed that while Quick Sort and Heap Sort provide the necessary scalability for managing tens of thousands of records, Heap Sort stands out as the most reliable candidate for mission-critical systems where predictable performance is paramount. Its ability to maintain a consistent logarithmic time complexity ensures that even under the most unfavorable data distributions, the system remains responsive. Conversely, the study highlighted the unique efficiency of Insertion Sort in niche scenarios. Although it performed poorly on large random arrays, its minimal overhead and exceptional speed on nearly sorted data make it an indispensable tool for incremental data updates, where new items are added to an already organized catalog. Ultimately, this research bridges the gap between theoretical complexity and practical implementation, allowing for the architecture of more robust and efficient product ranking engines.

9.2 Future Work

Building upon the foundational analysis presented in this study, several avenues for future research emerge that could further enhance the efficiency of data organization in digital platforms. One of the most promising directions is the exploration and implementation of hybrid sorting mechanisms. Modern computing environments increasingly rely on sophisticated algorithms like TimSort, which intelligently switches between different strategies based on the characteristics of the data fragments. Investigating how such hybrid models could be tailored specifically for e-commerce metadata could lead to a new standard in platform performance.

Furthermore, as the industry moves toward distributed computing and Big Data paradigms, there is a critical need to evaluate these algorithms in parallel processing environments. Future iterations of this project could focus on the implementation of multi-threaded sorting techniques that leverage modern multi-core CPU architectures or even GPU acceleration. Additionally, integrating machine learning models to predict the most efficient sorting

algorithm for a given batch of data represents a cutting-edge intersection of data science and classical algorithm design. By expanding the criteria beyond simple popularity scores to include multi-dimensional user behavior metrics, future research can provide a more holistic approach to real-time information retrieval and presentation.

10. References

- [1] Interview Cake, "Heapsort Algorithm," Interview Cake,.
Available: <https://www.interviewcake.com/concept/java/heapsort>
- [2] GeeksforGeeks, "Heap Sort," <https://www.geeksforgeeks.org/dsa/heap-sort/>, accessed Oct. 31, 2025.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [4] OpenGenus IQ, "Time and Space Complexity of Insertion Sort," *OpenGenus Computing Expertise & Legacy*, 2023. Available: <https://iq.opengenus.org/time-and-space-complexity-of-insertion-sort/>
- [5] GeeksforGeeks, "Insertion Sort Algorithm," *GeeksforGeeks Portal*, 2024. Available: <https://www.geeksforgeeks.org/insertion-sort-algorithm/>
- [6] K. Goel, P. Dwivedi, and O. Sharma, "Performance analysis of various sorting algorithms: Comparison and optimization," in 2023 11th International Conference on Intelligent Systems and Embedded Design (ISED), Dec. 2023.

11. Appendices

Appendix A: Complete Source Code

```
import pandas as pd
import time
import random
import sys
import matplotlib.pyplot as plt
import seaborn as sns

[ ]

from google.colab import files
import pandas as pd
uploaded = files.upload()

[ ]

sys.setrecursionlimit(20000)

def insertion_sort(arr):
    A = arr.copy()
    for i in range(1, len(A)):
        key = A[i]
        j = i - 1
        while j >= 0 and key < A[j]:
            A[j + 1] = A[j]
            j -= 1
        A[j + 1] = key
    return A

def heapify(arr, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2
    if l < n and arr[i] < arr[l]:
        largest = l
    if r < n and arr[largest] < arr[r]:
        largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    A = arr.copy()
    n = len(A)
    for i in range(n // 2 - 1, -1, -1):
        heapify(A, n, i)
```

```

def heap_sort(arr):
    A = arr.copy()
    n = len(A)
    for i in range(n // 2 - 1, -1, -1):
        heapify(A, n, i)
    for i in range(n - 1, 0, -1):
        A[i], A[0] = A[0], A[i]
        heapify(A, i, 0)
    return A

def quick_sort(arr):
    A = arr.copy()
    _quick_sort_helper(A, 0, len(A) - 1)
    return A

def _quick_sort_helper(arr, low, high):
    if low < high:
        pivot_index = _partition(arr, low, high)
        _quick_sort_helper(arr, low, pivot_index - 1)
        _quick_sort_helper(arr, pivot_index + 1, high)

def _partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

file_name = 'clean_dataset.csv'
column_name = 'rating'

try:
    df = pd.read_csv(file_name)
    full_dataset_list = df[column_name].dropna().tolist()
    print(f"Dataset is downloaded successfully! Total elements: {len(full_dataset_list)}\n")
except Exception as e:
    print("Dataset will be generated automatically. Please make sure your file is clean and free of errors.")

    full_dataset_list = [random.uniform(1.0, 1000.0) for _ in range(11000)]

sizes = [200, 500, 1000, 2200, 4300, 5500, 6700, 10000]

heap_times = []
quick_times = []
insertion_times = []
actual_sizes = []

for size in sizes:

```

```

for size in sizes:
    if size > len(full_dataset_list):
        break
    actual_sizes.append(size)

    sample_data = full_dataset_list[:size]

    # Heap Sort
    start = time.time()
    heap_sort(sample_data)
    end = time.time()
    heap_times.append((end - start) * 1000)

    # Quick Sort
    start = time.time()
    quick_sort(sample_data)
    end = time.time()
    quick_times.append((end - start) * 1000)

    # Insertion Sort
    start = time.time()
    insertion_sort(sample_data)
    end = time.time()
    insertion_times.append((end - start) * 1000)

results_df = pd.DataFrame({
    'Input Size (n)': actual_sizes,
    'Algorithm 1 (Heap) Time (ms)': heap_times,
    'Algorithm 2 (Quick) Time (ms)': quick_times,
    'Algorithm 3 (Insertion) Time (ms)': insertion_times
})

pd.options.display.float_format = '{:,.3f}'.format

print("Final Results:")
print(results_df.to_string(index=False))

```

```

plt.figure(figsize=(10, 6))

sns.lineplot(x=actual_sizes, y=heap_times, marker='o', label='Heap Sort (O(n log n))', color='blue', linewidth=2)
sns.lineplot(x=actual_sizes, y=quick_times, marker='s', label='Quick Sort (O(n log n))', color='green', linewidth=2)
sns.lineplot(x=actual_sizes, y=insertion_times, marker='^', label='Insertion Sort (O(n^2))', color='red', linewidth=2)

plt.title('Performance Comparison: Heap vs Quick vs Insertion Sort', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Dataset Size (n)', fontsize=12, fontweight='bold')
plt.ylabel('Execution Time (ms)', fontsize=12, fontweight='bold')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=11)

plt.tight_layout()
plt.show()

```

Appendix B: Sample Test Data

1	Reviewer ID	Profile Link	Country	Review Count	Review Date	Rating	Review Title	Review Text	Date of Experience
2	Eugene at	/users/66	US	1 review	2024-09-1	Rated 1 out of 5	A Store That	I registered	#####
3	Daniel oh	/users/5d	GB	9 reviews	2024-09-1	Rated 1 out of 5	Had multi	Had multi	#####
4	p fisher	/users/54	GB	90 reviews	2024-09-1	Rated 1 out of 5	I informed	I informed	#####
5	Greg Dunr	/users/62	AU	5 reviews	2024-09-1	Rated 1 out of 5	Advertise	I have bou	#####
6	Sheila Har	/users/5d	GB	8 reviews	2024-09-1	Rated 1 out of 5	If I could g	If I could g	#####
7	Tamesha f	/users/61	US	4 reviews	2024-09-1	Rated 1 out of 5	Terrible, I	Terrible yo	#####
8	Irene Rous	/users/59	US	30 reviews	2024-09-1	Rated 1 out of 5	Yet again	Amazon h	#####
9	Jacqueline	/users/66	US	2 reviews	2024-09-1	Rated 5 out of 5	I love ama	I love ama	#####
10	Robert Fie	/users/66	GB	1 review	2024-09-1	Rated 1 out of 5	I applied f	I applied f	#####
11	MH	/users/66	US	3 reviews	2024-09-1	Rated 5 out of 5	I had a gre	I had a gre	#####
12	Jimmie De	/users/66	JP	3 reviews	2024-09-1	Rated 5 out of 5	Every time	Every time	5-Sep-24
13	Michael M	/users/65	GB	1 review	2024-09-1	Rated 1 out of 5	Amazon a	I have no i	#####
14	Anita Jame	/users/66	US	2 reviews	2024-09-1	Rated 2 out of 5	Bad experi	Bought a F	#####
15	Mark	/users/66	US	1 review	2024-09-1	Rated 1 out of 5	Sadly	Sadly, Am	6-Sep-24
16	Amanda	/users/61	GB	4 reviews	2024-09-1	Rated 1 out of 5	Extremely	Extremely	#####
17	Nina Geor	/users/5b	GB	12 reviews	2024-09-1	Rated 1 out of 5	Right shar	My bank c	#####
18	paul stoke	/users/5b	GB	16 reviews	2024-09-1	Rated 1 out of 5	Fraudulan	I put onw	#####
19	Devon OM	/users/60	CA	1 review	2024-09-1	Rated 1 out of 5	Tired of de	Once agai	#####
20	Diana A	/users/60	US	6 reviews	2024-09-1	Rated 1 out of 5	AMAZON I	Amazon is	#####
21	JERRY KIRI	/users/66	US	1 review	2024-09-1	Rated 2 out of 5	I bought r	I bought r	3-Sep-24
22	Chris	/users/66	GB	10 reviews	2024-09-1	Rated 1 out of 5	δY'© shov	I think I'll b	#####
23	Guy Bulloc	/users/53	GB	15 reviews	2024-09-1	Rated 1 out of 5	Amazon re	Bought a /	#####
24	f a	/users/66	US	1 review	2024-09-1	Rated 1 out of 5	Amazon c	Amazon c	#####
25	Mara Fincl	/users/60	US	2 reviews	2024-09-1	Rated 1 out of 5	amazon is	amazon h	#####
26	Em	/users/66	US	1 review	2024-09-1	Rated 1 out of 5	They lie ab	They lie ab	#####
27	Jon	/users/64	US	1 review	2024-09-1	Rated 1 out of 5	Enter at y	I ordered 2	#####
28	Boston Bo	/users/66	ZA	8 reviews	2024-09-1	Rated 1 out of 5	Customer	Customer	#####
29	Xmasgoos	/users/5a	GB	12 reviews	2024-09-1	Rated 1 out of 5	No matter	No matter	#####
30	Peter Gray	/users/5f	GB	20 reviews	2024-09-1	Rated 2 out of 5	Diff to cor	Many orde	#####

Figure 5 Dataset before cleaning

1	rating	review_co	popularity	popularity	popularity_score	
2	1	1	1	0.69315	0.69315	
3	1	9	9	2.30259	2.30259	
4	1	5	5	1.79176	1.79176	
5	1	8	8	2.19722	2.19722	
6	1	4	4	1.60944	1.60944	
7	5	2	10	5.49306	5.49306	
8	1	1	1	0.69315	0.69315	
9	5	3	15	6.93147	6.93147	
10	5	3	15	6.93147	6.93147	
11	1	1	1	0.69315	0.69315	
12	2	2	4	2.19722	2.19722	
13	1	1	1	0.69315	0.69315	
14	1	4	4	1.60944	1.60944	
15	1	12	12	2.56495	2.56495	
16	1	16	16	2.83321	2.83321	
17	1	1	1	0.69315	0.69315	
18	1	6	6	1.94591	1.94591	
19	2	1	2	1.38629	1.38629	
20	1	10	10	2.3979	2.3979	
21	1	15	15	2.77259	2.77259	
22	1	1	1	0.69315	0.69315	
23	1	2	2	1.09861	1.09861	
24	1	1	1	0.69315	0.69315	
25	1	1	1	0.69315	0.69315	
26	1	8	8	2.19722	2.19722	
27	1	12	12	2.56495	2.56495	
28	1	5	5	1.79176	1.79176	
29	1	2	2	1.09861	1.09861	
30	1	1	1	0.69315	0.69315	

Figure 6 Dataset after cleaning